If that unique point of failure concerns the interaction between two objects (or sometimes, the interaction between two methods of the same object), then an integration test is called for. If the unique point of failure is the internal logic of a single object, then that condition is better covered with a unit test.

In a Rails context, the following are fodder for system tests:

- The interaction between a controller and the model or other objects that provide data
- The interaction between multiple controller actions that comprise a common workflow—for example, presenting a form, filling it out, and then submitting it
- Certain security issues that involve the interaction between a user state and a particular controller action

The following things, generally speaking, are *not* best tested using system tests, or any other kind of integration test. Use unit tests instead for these:

- Special cases of business logic, such as what happens if data is nil or has an unexpected value
- Error cases, unless it genuinely results in a unique user experience
- Internal implementation details of business logic

Two kinds of problems happen when you use system tests to cover things that are better done in unit tests. The first is speed. System tests are slower—not because the tools themselves are slow, but because the tests are winding their way through the Rails stack to get to the method you're interested in. Because the individual tests each cover such a wide range of code, an entire suite of integration tests makes it hard to diagnose where failures are occurring when they occur.

The second is precision. Because the system test is not making any assertions until after the internal logic has executed, it's often hard to piece together what went wrong. Often the way to deal with this is to have a failing feature test trigger the writing of a unit test.

Outside-in Testing

The process you'll use to manage the feature tests is sometimes called *outside-in testing*—you use Capybara to write a test from the outside and use that test to drive your unit tests. In the same way that TDD uses a failing unit test to drive code, outside-in testing uses a failing acceptance test (or a failing line in an acceptance test) to drive the creation of unit tests.